

Task: JavaScript-Enhanced Form Page

Task Objective

Extend the responsive form page built in Task 6.3D by replacing HTML validation with JavaScript validation and adding dynamic JavaScript features that improve the user experience.

Task Description

In Task 6.3D, you built a responsive form integrated into your own website using HTML validation attributes. For this Distinction task, remove all HTML5 validation attributes from the form and implement a fully JavaScript-driven validation system with progressive enhancement features.

The form must remain part of your website. The exact fields and enhancements will depend on your website concept, but all JavaScript techniques must meet the requirements below.

Requirements

Part 1 — Replace HTML Validation with JavaScript

- Remove all built-in HTML5 validation constraints (such as required, pattern, minlength) and implement all validation rules in JavaScript.
- Implement a validate() function that runs when the form is submitted via modern event listeners.
- Use event.preventDefault() to stop the form from reloading or submitting normally.
- Validate every form field using JavaScript rules.
- Show inline validation feedback for each field.
- Track validation status using a boolean flag; allow the process to finish cleanly only if all fields pass.
- Use regular expressions to validate at least two fields that suit your website concept.

Part 2 — Dynamic JavaScript Enhancements

In addition to basic validation, implement at least three (3) of the following JavaScript enhancements. Choose the ones that best suit your website concept and justify your choices in your written reflection.

Enhancement	Description
A — Live Character Counter	For a <code><textarea></code> field, display a live character count that updates as the user types
B — Real-Time Field Validation	Validate one or more fields as the user types or leaves the field, using input or blur events — not just on submit
C — Dynamic Field Show/Hide	Show or hide a form field based on a user's selection using an onchange event
D — Submission Summary	On successful validation, dynamically display a summary of the submitted data in the page instead of actually submitting
E — Password Strength Indicator	If your form includes a password field, display a live password strength indicator
F — Reset with Confirmation	Replace the default reset button with a JavaScript-controlled reset that asks the user to confirm before clearing the form

Important Note: When removing HTML5 validation:

- ✓ Add **novalidate** to your `<form>` tag to disable native browser validation popups.
 - ✓ Keep **aria-required="true"** on required fields for accessibility so screen readers know they are mandatory.
 - ✓ Use **addEventListener('submit', ...)** with `event.preventDefault()` in your JavaScript (avoid inline HTML onsubmit attributes).
 - ✓ **For real-time validation (Enhancement B)**, you may show green success feedback immediately when a field meets all criteria as the user types.
-

Technical Requirements

Code Quality

- Keep the JavaScript well organised and commented.
- Use meaningful variable and function names.
- Separate validation logic from DOM updates where possible.
- Ensure there are no console errors.

Validation Messages

- Show inline feedback for every field.
- Use red for invalid messages.
- Use green for valid messages, but only after successful validation or on blur.
- Make messages clear and specific, not just "Invalid".

Regular Expressions

- Use regex for at least two fields.
- Add comments explaining what each regex checks.
- Test the patterns in browser DevTools.

Responsiveness and Integration

- Make sure validation messages and enhancements work on both mobile and desktop.
- Keep the form visually and functionally integrated with your Task 6.3D website.
- Update your existing stylesheet only; do not create a separate CSS file.
- The form is processed entirely on the same page; no backend is required.

Checklist

- ☐ All HTML5 validation constraints (required, pattern, etc.) removed (keeping aria-* attributes for accessibility)
- ☐ validate() runs on submit hook with event.preventDefault()
- ☐ Inline validation messages for all fields
- ☐ RegExp used for at least two fields, with explanatory comments
- ☐ At least 3 dynamic enhancements implemented
- ☐ Enhancements work seamlessly on mobile and desktop layouts
- ☐ No console errors present during testing
- ☐ Form visually integrated with Task 6.3D website

- ☐ Demo recording link submitted and accessible to your marker AND Unit Chair
 - ☐ Written reflection submitted (150–200 words)
-

Deliverables

- **Updated HTML Form File**
 - HTML5 validation constraints removed.
 - Inline feedback containers (or <div> labels) added.
 - Target IDs or classes added for JavaScript selectors.
- **JavaScript File**
 - Form submission event logic.
 - Regex patterns documented with code comments.
 - At least three dynamic enhancements.
 - Clean, well-commented code.
- **Updated CSS File**
 - Validation styling for red (invalid) and green (valid) feedback states.
 - Styling setups for chosen enhancements.
 - Responsive layout adjustments.
- **Demo recording link accessible to your marker AND Unit Chair.**

A 2–4 minute screen recording showing:

 - An invalid submission displaying clear red error messages.
 - A valid submission showcasing a success state or summary.
 - All three chosen enhancements working interactively.
 - A mobile responsive view showing readable feedback layout.
 - Developer browser console showing zero errors.

- **Written Reflection**

A 150–200 word PDF explaining:

- Why you chose your three specific enhancements.
- How your chosen regex patterns work.
- Key troubleshooting challenges you faced and how you solved them.
- How the updated form layout helps your target users.